

SKETCH IN AIR

PROJECT REPORT

Submitted by

Mallikharjuna K
17MIS1031

in partial fulfillment for the award of the degree of

Master of Technology

in

Software Engineering - (5 Year Integrated Programme)



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

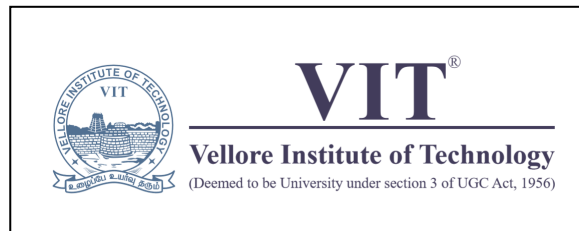
Vellore Institute of Technology

Vandalur - Kelambakkam Road, Chennai - 600 127

May - 2022

SKETCH IN AIR

i



School of Computer Science & Engineering

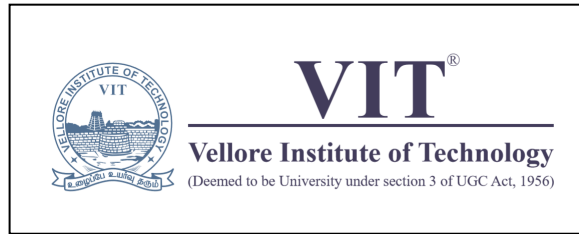
DECLARATION

I hereby declare that the undertaking project entitled **Sketch In Air** submitted by using me to the School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, 600 127, in partial success of the necessities of the award of the diploma of Master of Technology in Software Engineering (5 yr Integrated Programme) and as section of SWE4009 – Capturestone Project is a bona-fide file of the work carried out via me below the supervision of **Dr. Vedhapriyavadhana R** man. I in addition declare that the work mentioned in this project, has now not been submitted and will no longer be submitted, both in section or in full, for the award of any different diploma or diploma of this institute or of any different institute or University..

Place: Chennai,
Date:25/04/2022.

Signature of Candidate

K. Mallikharjuna



School of Computer Science & Engineering

CERTIFICATE

This is to certify that the document entitled Sketch In Air is organized and submitted by way of Mallikharjuna K (Reg. No. 17mis1031) to Vellore Institute of Technology, Chennai, in partial achievement of the requirement for the award of the diploma of Master of Technology in Software Engineering (5 12 months Integrated Programme) and as phase of SWE4099 – Capturestone Project is a bona-fide file carried out underneath my guidance. The undertaking fulfills the necessities as per the rules of this University and in my opinion meets the crucial requirements for submission.

Guide/Supervisor

Name: Dr. Vedhapriyavadhana R
Date: 03/05/2022

HoD

Name: Dr. Asnath Vicky Phamila Y
Date:

Examiner

Name:
Date:

Examiner

Name:
Date:

Acknowledgement

I obliged to provide my grasp to a variety of humans barring whom I should no longer have executed this thesis successfully.

I would like to region on report my deep experience of gratitude and thanks to my inner guide Prof. Vedhapriyavadhana R, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, whose esteemed assist and vast preparation prompted me to whole the challenge successfully.

I would like to thank our HoD Dr. Asnath Victy Phamila Y, School of Computer Science and Engineering (SCOPE) and Project Coordinator Dr. Rabindra Kumar Singh, Vellore Institute of Technology, Chennai, for their precious help and encouragement to take up and whole this thesis.

Special point out to our Dean Dr. Ganesan R , Associate Dean Dr. S. Geetha, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, for motivating us in each factor of software program engineering.

I thank our administration of Vellore Institute of Technology, Chennai, for enabling me to use the library and laboratory resources. I additionally thank all the college contributors for giving me the braveness and the energy that I wished to entire my goal. This acknowledgment would be incomplete besides expressing the total hearted thanks to my household and buddies who stimulated me throughout the direction of my work.

Mallikharjuna K
17MIS1031

Abstract

Writing in air has been one of the most captivating and difficult lookup areas in discipline of picture processing and sample cognizance in the current years. It contributes immensely to the development of an automation procedure and can enhance the interface between man and desktop in several applications. Several lookup works have been focusing on new methods and techniques that would decrease the processing time whilst offering greater awareness accuracy. Object monitoring is viewed as a necessary undertaking inside the discipline of Computer Vision. The invention of quicker computers, availability of less expensive and precise nice video webcams and needs of computerized video evaluation has given recognition to object monitoring techniques. The mission takes benefit of this hole and focuses on growing a graph in air as nicely as an emojinator. This undertaking is a reporter of occasional gestures and emotions (through emojis). It will use laptop imaginative and prescient to hint the direction of the finger or user's precise action. The layout in air can be used to draw some thing in the air and the emojinator can be used in speech-to-text structures in order for gadget to take emojis as inumpyut.

Keywords

Air Writing, Object Detection, Real-Time Gesture Control System, Computer Vision.

Contents

Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
1 Introduction	7
1.1 Background -----→	8
1.2 Statement -----→	9
1.3 Motivation -----→	9
1.4 Challenges -----→	10
2 Planning & Requirements Specification	12
2.1 System Planning -----→	12
2.2 Requirements -----→	15
2.2.1 User requirements -----→	15
2.2.2 Non-Functional requirements -----→	15
2.3 System Requirements. -----→	15
2.3.1 Hardware Requirements -----→	15
2.3.2 Software Requirements -----→	16
3 *System Design	17
4 *Implementation of System	19
5 *Results & Discussion	21
6 *Conclusion and Future Work	22
*References	23
*Appendix <Sample code, snapshot etc.>	24

1 Introduction

In the time of computerized world, conventional specialty of update is being supplanted by advanced workmanship. Computerized craftsmanship alludes to plumes of articulation and transmission of kind with advanced structure. Relying on ultramodern insight and innovation is the unmistakable qualities of the computerized manifestation. Conventional craftsmanship alludes to the sort which is made before the advanced workmanship. From the altruist to examine, it can just be isolated into visual workmanship, sound craftsmanship, general media craftsmanship and general media fanciful workmanship, which incorporates writing, oil, structure, armature, music, cotillion, show and other plant of workmanship.

PC visionary workmanship and customary or conventional craftsmanship are interrelated and reliant. Social improvement is definitely not a group's will, yet the states of mortal life are the really main thrust at any rate. A similar circumstance occurs in craftsmanship. In the current conditions, computerized craftsmanship and customary workmanship are comprehensive of the cooperative state, so we really want to comprehend the early on information on the structure between advanced workmanship and conventional craftsmanship completely.

The customary way incorporates pen and paper, chalk and board arrangement of notice. The fundamental finish of computerized craftsmanship is of raising hand motion acknowledgment framework to carefully compose. Computerized craftsmanship incorporates various approaches to writing like by utilizing console, contact screen face, advanced pen, pointer, utilizing electronic hand gloves, and so on. In any case, in this framework, we're utilizing hand signal acknowledgment with the utilization of profound information by utilizing python programming, which makes normal business among man and machine. With the headway in innovation, the need of advancement of normal' mortal - PC business (HCI)' frameworks to supplant customary frameworks is adding fleetly.

Addicts are utilized to draw on virtual oils utilizing mouse or the mouse cushion in their PCs yet we're it that is enough significant imperceptible to propose an oil. This is an initial plan laid on PC vision made in OpenCV Python which empowers the stoner to

Draw on their framework screen by attracting air with an objective, rather the tips of your gallette, which is followed by the PC webcam.

The end is to follow the objective first, and furthermore its mix and be reasonable to imitate its way on the screen. This is finished utilizing OpenCV poisons like Gaussian haze. The place of the objective is followed, the picture is concealed and the focal point of the objective is determined. Likewise the way of the focal point of the objective is drawn on the screen.

This is the starting thought of what's wanted to be finished. A few different elements to be added will incorporate a variety range, and different highlights of an exemplary Paint activity. The compass for different progressions could incorporate integrating a model to decipher what's composed, atleast figures.

Emoticons are ideograms and smileys utilized in electronic dispatches and web sprinters. Emoticon live in brilliant stripes, including looks, normal articles, places and kinds of destruction, and beasties. They're significant like emojis, yet emoticon are genuine film rather of typographics. Emojinator is an emoticon sensor programming which can fete and order different hand emoticons.

1.1 Background

The creators explored a consistent fingertip identification in outlines caught from the inexorably nicely regarded wearable gadget, savvy glasses. The egocentric-view fingertip discovery and personality acknowledgment proposed in this paper can be utilized to make a wise method to contributing texts.

Unity3D is utilized to fabricate a manufactured dataset with pointing motions from the first-individual viewpoint. Following that, an altered Mask Regional Convolutional Neural Network (Mask RCNN) is proposed, comprising of a locale primarily based CNN for finger identification and a three-topped and patched CNN for fingertip area.

The pace closed with the aid of the writers is sufficiently excessive to empower consistent "airwriting", the place consumers can compose characters in the air to enter texts or orders whilst carrying savvy glasses. The characters are regularly perceived with the aid of a ResNet-based CNN from the fingertip directions

1.2 Statement

Text Writing in Air The authors in presented a constant video based pointing technique which permits drawing and composing of English text over air before versatile webcam. The proposed technique has two principle assignments: first it tracks the hues fingertip in the video casings and afterward apply English OCR over plotted pictures to perceive the composed characters.

Besides, the proposed strategy gives a characteristic human-framework cooperation in such manner that it doesn't need keypad, pointer, pen or glove and so forth for character inumpyut. The proposed framework is a product based approach and appropriately exceptionally straightforward, quick and simple.

It doesn't need sensors or any equipment instead of webcam and formality. The main disadvantage is that it is variety delicate so that presence of any red tone behind the scenes prior to beginning the person composing can prompt misleading outcomes.

1.3 Motivation

Users are used to draw on virtual drawings using mouse or the mouse pad in their laptops but we are proposing a drawing that's pretty much invisible. This is a basic project based on computer vision made in OpenCV Python which enables the user to draw on their system screen by drawing in air with a target, preferably the tips of your finger, which is tracked by the computer webcam.

The aim is to track the target first, and then its motion and be able to replicate its path on the screen. Air drawing is implemented using open cv, it detects colour of the pen adds

required colour on the drawing. Different image processing techniques of open cv are also taken in to account.

1.4 Challenges

Identification of RGB Colors:

In this stage, the 3 layered colors are separated from the information picture. All the variety pictures on screens, for example, TVs, PC, screens, PCs and versatile screens are delivered by the blend of Red, Green and Blue light. Every essential variety takes an escalated esteem 0 (most reduced) to 255 (most noteworthy). While blending 3 essential tones at various power levels an assortment of varieties are delivered. For Example: If the power worth of the essential tones is 0, this straight mix relates to dark. Assuming the force worth of the essential tones is 1, this straight mix relates to white. Index=["variety", "color_name", "hex", "R", "G", "B"] Calculate least separation from facilitates: The base distance is determined by considering moving towards the beginning point from all tones to get the most matching variety.

Fixing Mask:

Picture inumpyainting is the method involved with eliminating harm, like clamors, strokes, or text, on pictures. It is especially valuable in the rebuilding of old photos which could have scratched edges or ink spots on them. These can be carefully taken out through this technique.

Point Identification/Detection:

In this characterization of trackers, the fundamental overlaying is checked practicing a rectangular design to show the area of the article we need to follow. The organization is furthermore went with in past edges practicing the accompanying calculation. In most extreme legitimate tasks, these trackers are utilized related to a structure record.

Getting oof the mask-noise/gray marks:

As of now It's presently not requested that you get a best cowl in the when improve and its to have some clatter like white spots in the image, we can discard this disturbance with morphological molding in this progression. Presently I am practicing a variable namedload_from_disk to pick to hill the reach fluctuate from circle or might I want to utilize a many redid quotes.

Making line in Video Feed:

Regardless, we take an appear to be at assuming the previous center element is 0, and that presents up to nothing is connected in the previous edge. In any case, in thenon-stop edge, tolerating that some component is recognized, we generally unfeignedly know about the comportments of the diverseness affirmation link tos. So we checked in which link to the middle indication of the design is extraordinary first. What is further, along these lines set the diverseness likewise. The central link to is for clearing the material, this is the defense assuming the center part is perceived in the conclusive link to, we reset the fabric to a fresh start the utilization of bottoms () work.

By and with the helpful asset of we take an appear at in the event that the previous center variable isn't 0, and that deduces a construction is distinguished in the previous edge, moreover we describe an outline between the previous center component and thenon-stop center point. From that point forward, we substitute the previous center variable to thenon-stop center component.

2 Planning & Requirements Specifications

2.1 System Planning

Import necessary packages.

To IMPLEMENT this task, we really want a few bundles, OpenCV and Numpy, matlab.lib. also, and so forth .So first we've to import these.

Read frames from a webcam

In the first place, we made a Video taking web object and characterized the item as capture.

capture.read() work peruses each edge from the webcam.

opencv.flip() flips the edge. The contentions are inumpyut picture/edge and pivot. 0 is for flip upward, and 1 for flip evenly.

opencv.imshow() work shows outline in another window.

opencv.waitKey(1) == ord('q') keep open the window

Create the drawing window

Then, at that point, we make a vacant fresh start a similar size as the webcam outline. `Numpy_zeros()` make a lattice containing all zeros's.

Detect the Blue color

We need to change the pencil tone during painting, That's the reason we need to make a few link tos in the casing. Utilizing those link tos we can choose the pencil tone during painting progressively.

Utilizing `opencv.rectangle()`, we draw a few square shapes of various varieties that we've characterized previously.

`opencv.putText()` draws texts on a casing. Utilizing this captureacity we put every one of the names of the tones to their comparing square shape box

Draw on the Video Feed

To begin with, we check if the stored x,y coordinates is 0, and that amounts to nothing is distinguished in the past casing.

In any case, in the ongoing edge, assuming there is something recognized, one definitely have knowledge in the directions of a variety determination link tos as any count. So we checked in which link to the middle mark of the form is recognized first. And afterward set the variety likewise.

The principal link to is for clearing the material, that is the reason in the event that the middle point is recognized in the main link to, we reset the material to a fresh start utilizing `numpy.zeros()` work.

Utilizing `opencv.cvtColor()` work we modify the material to grayscale picture.

Then utilizing `opencv.threshold()` we make a double cover of the material. Of course we modify the double cover to BGR variety space. Since to add two pictures, the state of the two pictures should be something similar. `opencv.bitwise_and()` is valid on the off chance that the two pixels are more prominent than nothing.

2.2 Requirements

2.2.1 User Requirements

This Project is able to address the basic to intermediate level to core understanding of object detection is done and point calculation based on frames and x,y coordinates.

2.2.2 Non-Functional Requirements

People should know the basic object detection (which depends on python scripts) process to fully understand this project and to use efficiently in various ways.

2.3 System Requirements

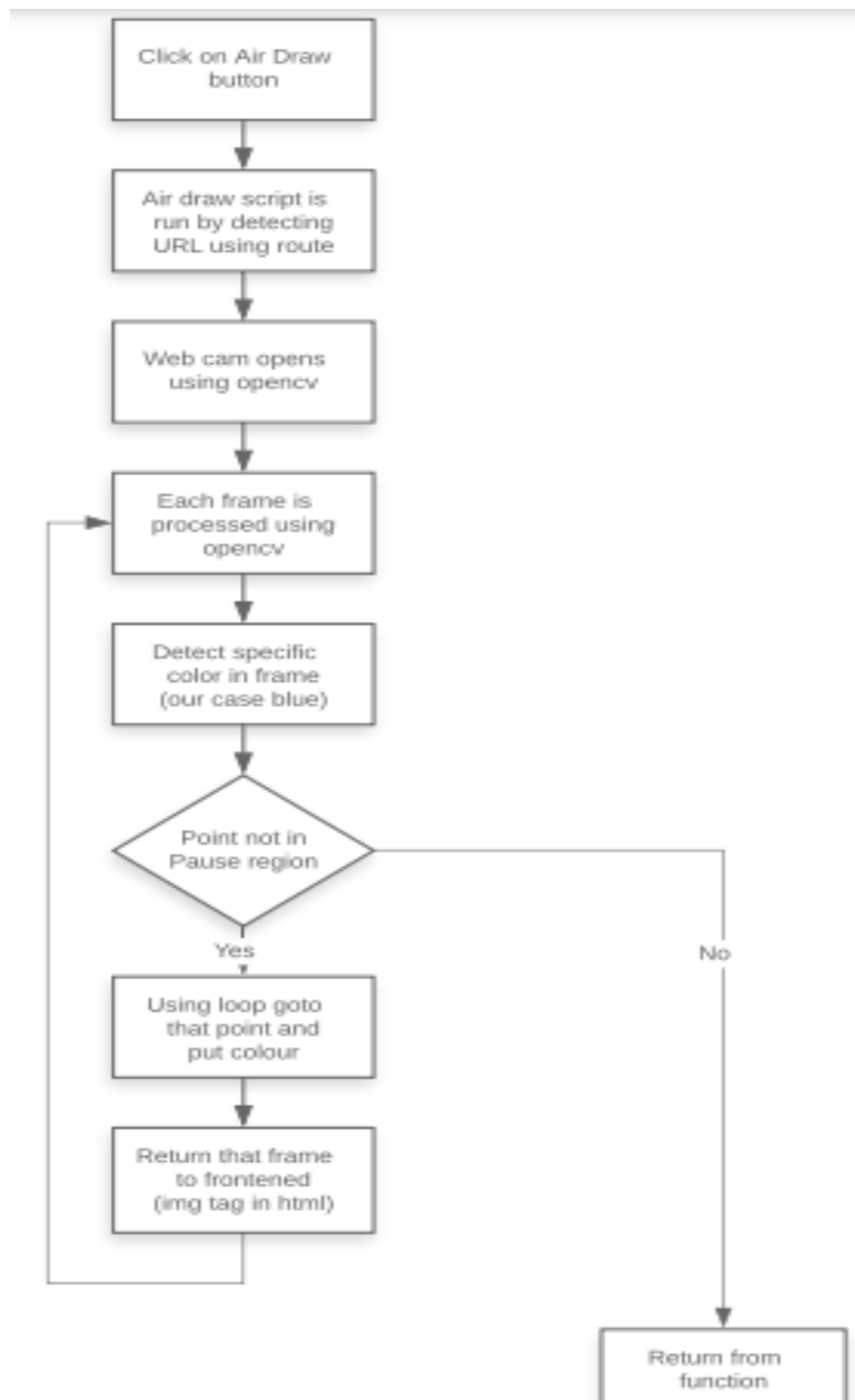
2.3.1 Hardware Requirements

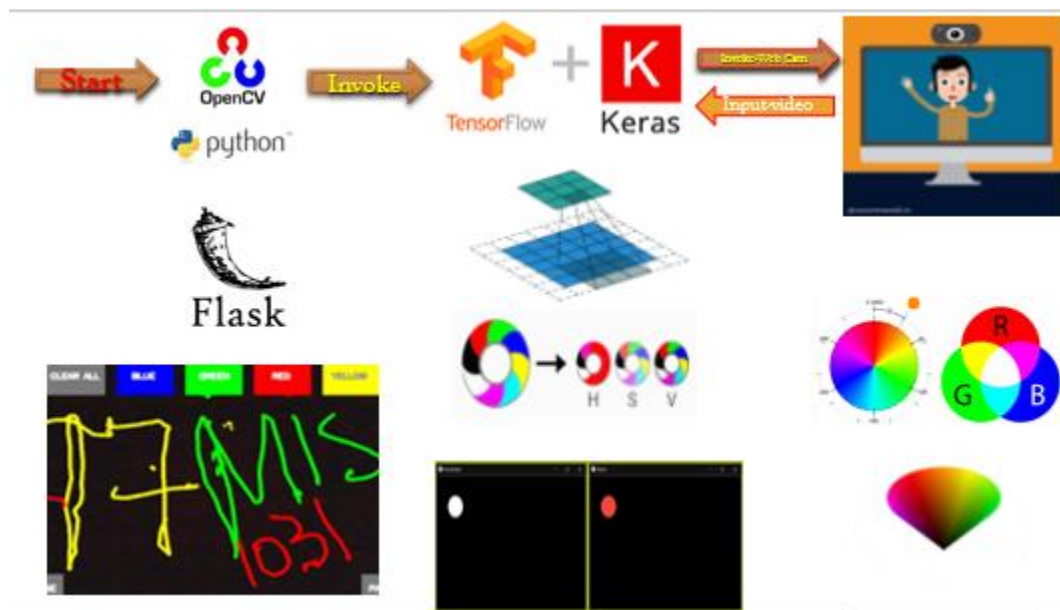
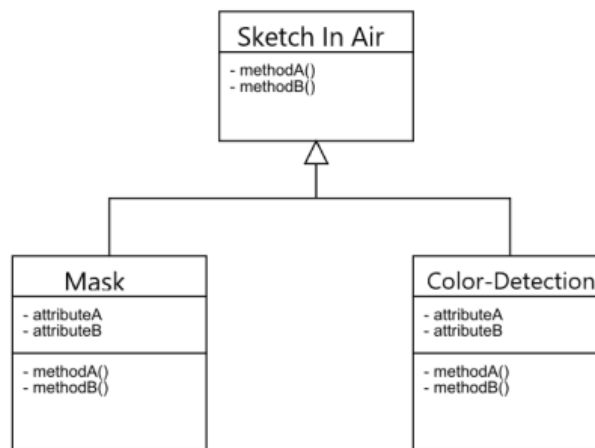
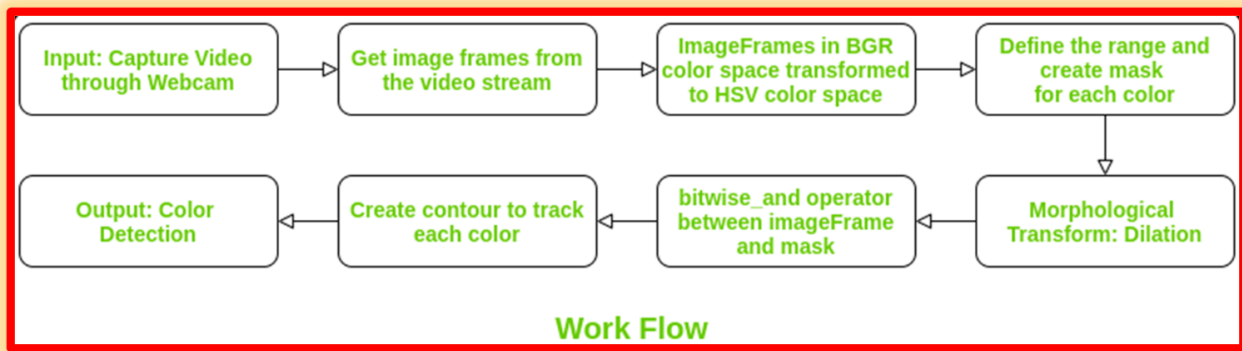
- NVIDIA graphics card & respective drivers
- CUDA driver v11.1 (for tensorflow)
- cuDNN v8.1
- Inbuild webcam
- Ram-8gb
- 2gb-disk-space
- Any color monitor.

2.3.2 Software Requirements

- Python v3.10.1
- Flask
- openCV-opencv
- Tensorflow-latest
- Keras
- HTML
- Bootstrap

3 System Design





4 Implementation of system

STEPS IN DETAIL:

- A. Colour Variety Tracking of Object at fingertip. Above all else, The approaching picture from the webcam is to be changed over completely to the HSV variety space for distinguishing the colorful article at the tip of cutlet.
- B. The beneath regulation grain switch/link/linkes the approaching picture over completely to the HSV space, which is authentically appropriate and consummate variety space for Colortracking. Now, We'll make the Trackbars to orchestrate the HSV values to the necessary scope of shade of the kaleidoscopic article that we've set at ourfinger.ow, When the trackbars are arrangement, we will get the realtime esteem from the trackbars and produce range.
- C. This reach is a numpy structure which is utilized to be passed in the function `opencv.inrange ()`. This captureacity returns the Mask on the diverse item. This Mask is a highly contrasting picture with white pixels at the place of the asked variety.
- D. Figure Discovery of the Mask of Color Object Now, After identifying the Mask in Air Drawing, Now is an ideal opportunity to recognize its middle situation for taking a stand. Then, at that point, In the beneath Snippet of Code, We're playing out a few morphological procedure on the Mask, to make it liberated from pollutions and to smoothly descry figure.
- E. Defining the Boundary utilizing the place of Contour Now Comes the genuine sense behind this Computer Vision configuration, We'll frame a python deque (An information Structure). The deque will store the place of the figure on each back to back edge and we will utilize these put away focuses to make a line utilizing OpenCV drawing captureacities.
- F. Presently, we will utilize the place of the figure to go with choice, to tap on a link to or we need to draw on the distance.
- G. We've organized a portion of the link tos on the highest point of Drawing, assuming the pointer comes into their area, we will start their framework. We've four link tos on the oil, drawn utilizing OpenCV.
- H. The `Clear()` function Which clears the screen by emptying the deque. Red Changes the marker to red variety utilizing variety cluster. Green Changes the

marker to Green variety utilizing variety exhibit. Unheroic Changes the marker to Unheroic variety utilizing variety exhibit.

- I. Blue Changes the marker to Blue variety utilizing variety cluster. Likewise, to stay away from outline when figure is absent, We'll Put an extra condition which will catch that second.
- J. Drawing the focuses Now we will draw every one of the focuses on the positions put away in the deque, with isolated variety.

ALGORITHM

- 1) Begin perusing the edges and modify the caught casings to HSV variety space.(Easy for variety recognition)
- 2) Set up the material edge and put the separate ink link tos on it.
- 3) Change the trackbar values for tracking down the veil of shaded marker.
- 4) Preprocess the veil with morphological operations.(Eroton and expansion)
- 5) Perceive the shapes, concentrate on the middle directions of biggest design and continue to shop them in the group for current edges .(Arrays for drawing centers around material)
- 6) Finally draw the spotlights set aside in show on the housings and material.

5 Results & Discussion

- Once the Flask web app is started, the python code invokes the OpenCv which reads the Video Feed in real time.
- From the video feed we take the pixels/frames to make a virtual pen using counter pointer.
- We achieve the above functionality by morphological operations and dilation principles in a mask using tensorflow for rendering the video inumpyut and real time pixel operations.
- As we get the pointer or color point detection we make an array of frames to store every frame the point moves using the location of x,y coordinates.
- Now every point is marked as previous x,y location after the pointer changes or moves from its position.
- Also the mask postion where ever the pointer moves we make the mask_not function as TRUE so when we make this mask postion as FALSE, we remove the array values which are visible lines. What we call this function is Eraser.
- And we can clear the screen by just making the line array as empty so all the previous frame (x,y coordinates) locations non-exist.

6 Conclusion and Future work

The framework can possibly challenge conventional show and instructing techniques. It will likewise fill an incredible need in aiding particularly abled individuals convey without any problem. Indeed, even senior residents or individuals who find it hard to utilize consoles will ready to easily utilize framework.

Expanding the usefulness, framework can likewise be utilized while video calls in no time. The framework will be a fantastic programming for brilliant wearables utilizing which individuals could more readily associate with the advanced world. Expanded Reality can make text wake up. There are a few constraints of the framework which can be worked on from now on.

Right off the bat, utilizing a penmanship recognizer instead of a person recognizer will permit the client to compose word by word, making composing quicker. Also, handgestures with a delay can be utilized to control the constant framework as done by [1] as opposed to utilizing the quantity of fingertips.

Thirdly, our framework some of the time perceives fingertips behind the scenes and significantly alters their state. Air-composing frameworks ought to just submit to their lord's control signals and ought not be misdirected by individuals around. Likewise, we can utilize the EMNIST dataset, which is an appropriate dataset. Forthcoming article identification calculations, for example, YOLO v3 can further develop fingertip acknowledgment exactness and speed. Later on, progresses in Artificial Intelligence will upgrade the effectiveness of SKETCH IN AIR.

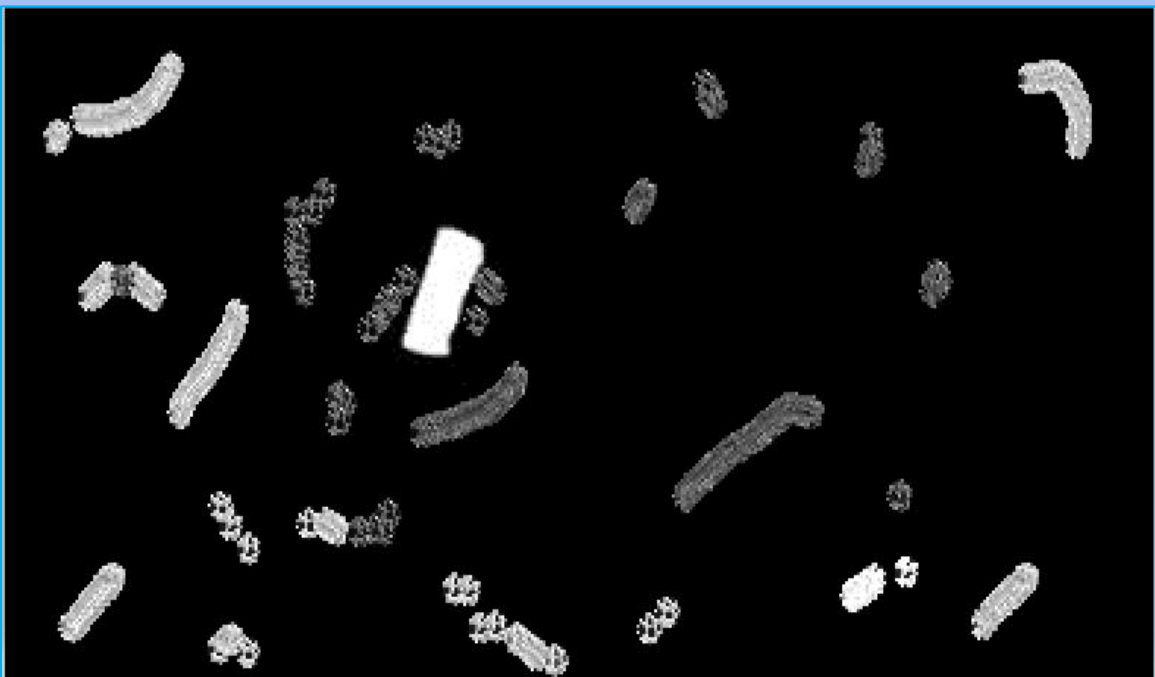
References

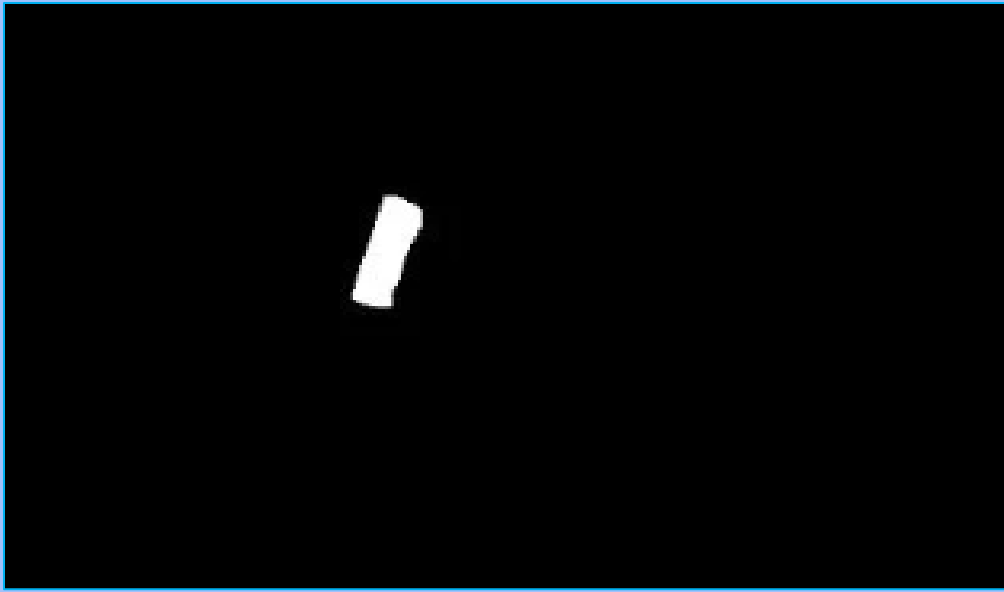
- 1] V. Joseph, A. Talpade, N. Suvarna and Z. Mendonca, "Visual Gesture Recognition for Text Writing in Air," 2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS), 2018, pp. 23-26, doi: 10.1109/ICCONS.2018.8663176
- 2] T. Sharma, S. Kumar, N. Yadav, K. Sharma and P. Bhardwaj, "Air-swipe gesture recognition using OpenCV in Android devices," 2017 International Conference on Algorithms, Methodology, Models and Applications in Emerging Technologies (ICAMMAET), 2017, pp. 1- 6, doi: 10.1109/ICAMMAET.2017.8186632.
- 3] J. Patel, U. Mehta, K. Panchal, D. Tailor and D. Zanzmera, "Text Recognition by Air Drawing," 2021 Fourth International Conference on Computational Intelligence and Communication Technologies (CCICT), 2021, pp. 292-295, doi: 10.1109/CCICT53244.2021.00061.
- 4] Chen Y-H, Huang C-H, Syu S-W, Kuo T-Y, Su P-C. Egocentric-View Fingertip Detection for Air Writing Based on Convolutional Neural Networks. Sensors. 2021; 21(13):4382. <https://doi.org/10.3390/s21134382>
- 5] E. Ohn-Bar and M. M. Trivedi, "Hand Gesture Recognition in Real Time for Automotive Interfaces: A Multimodal Vision-Based Approach and Evaluations," in IEEE Transactions on Intelligent Transportation Systems, vol. 15, no. 6, pp. 2368-2377, Dec. 2014, doi: 10.1109/TITS.2014.2337331.
- 6] Baig, Fasial & Khan, Muhammad & Beg, Saira. (2013). Text writing in the air. Journal of Information Display. 14. 10.1080/15980316.2013.860928.
- 7] K. Li and X. Zhang, "A new fingertip detection and tracking algorithm and its application on writing-in-the-air system," 2014 7th International Congress on Image and Signal Processing, 2014, pp. 457-462, doi: 10.1109/CISP.2014.7003824.
- 8] Y. Chen, P. Su and F. Chien, "Air-Writing for Smart Glasses by Effective Fingertip Detection," 2019 IEEE 8th Global Conference on Consumer Electronics (GCCE), 2019, pp. 381-382, doi: 10.1109/GCCE46687.2019.9015389.
- 9] C. T. Tolentino, A. Uy and P. Naval, "Air Drums: Playing Drums Using Computer Vision," 2019 International Symposium on Multimedia and Communication Technology (ISMAT), 2019, pp. 1-6, doi: 10.1109/ISMAT.2019.8836175.

Appendix <Sample code, snapshot etc.>

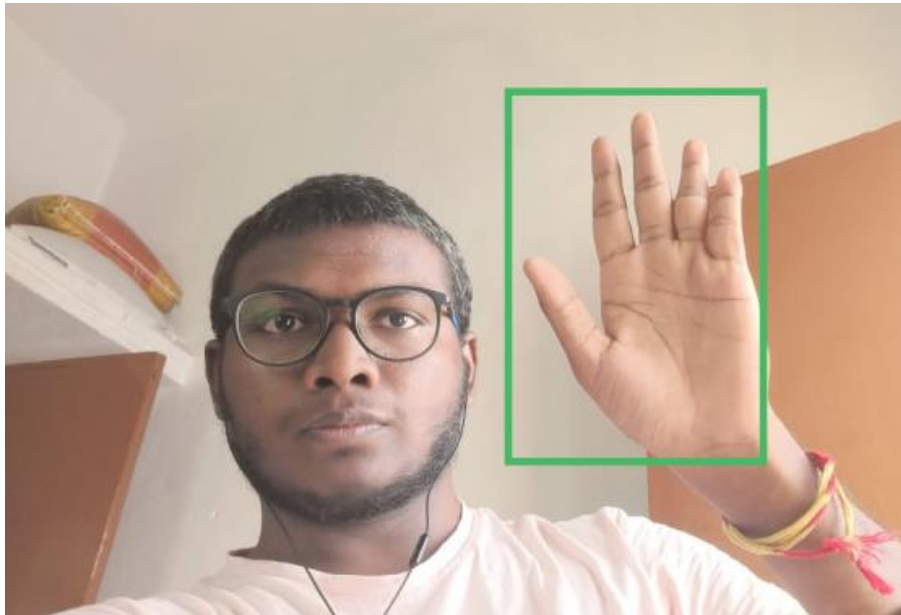
Results Snapshots

Fixing Mask position:

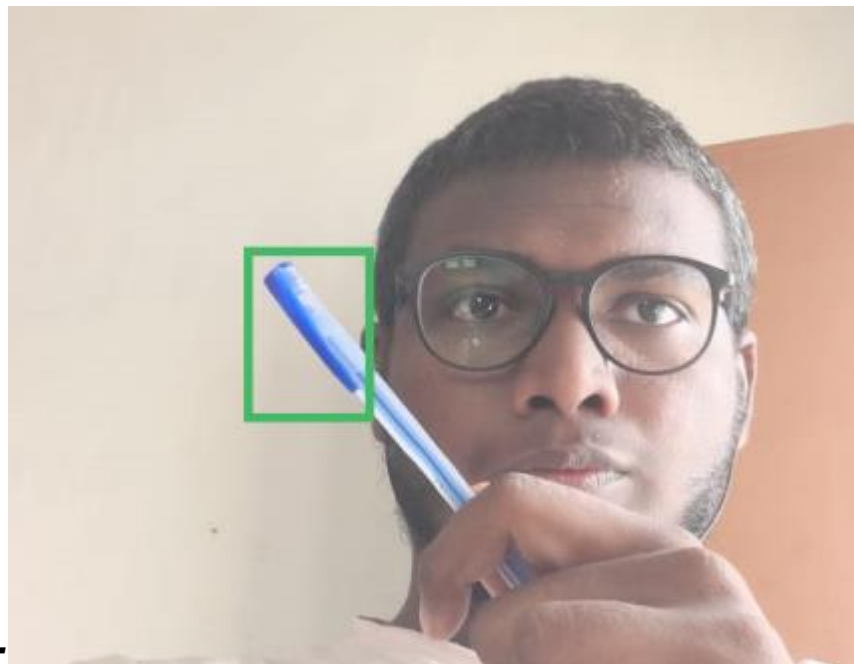


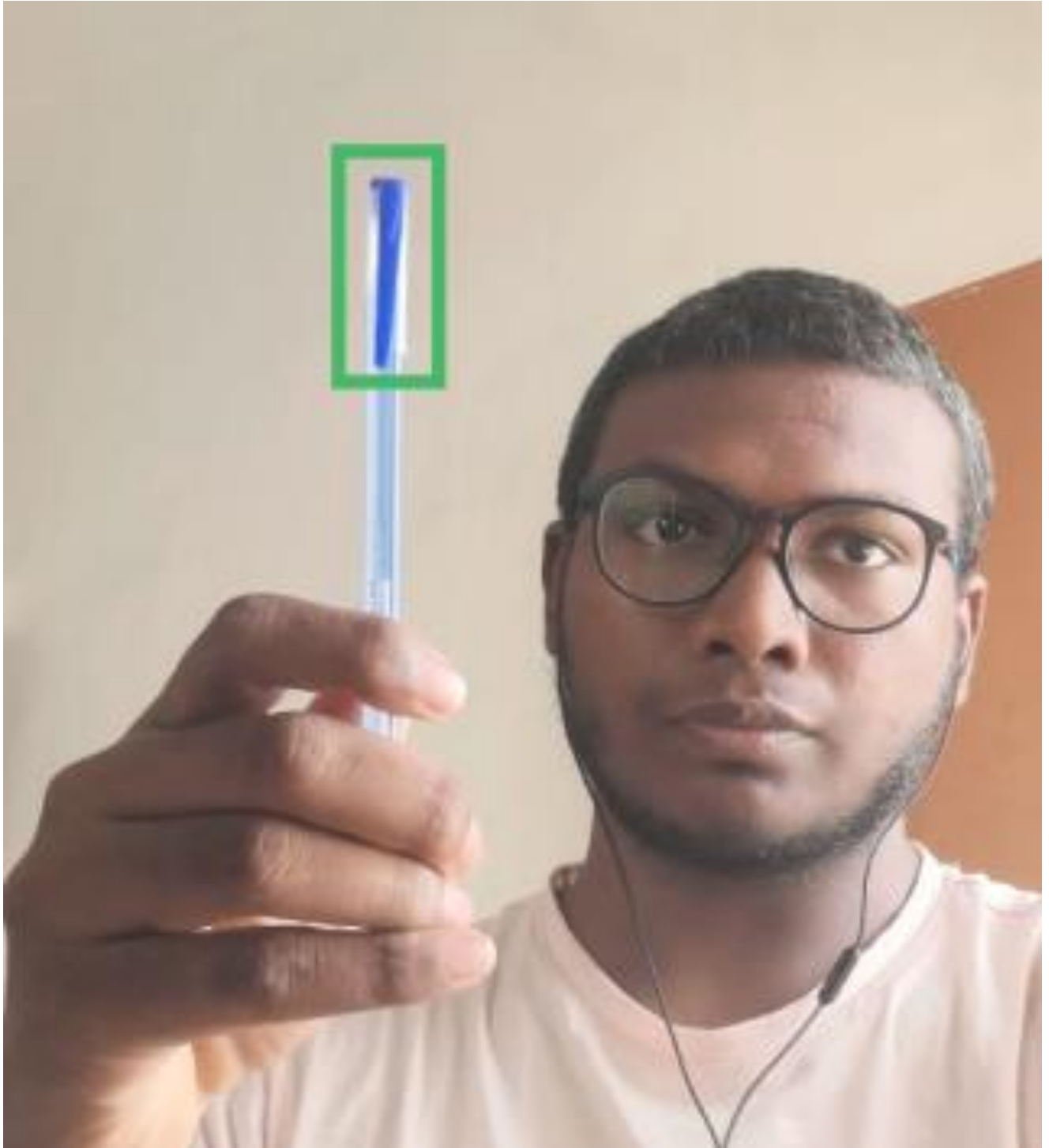


Object/hand detection:

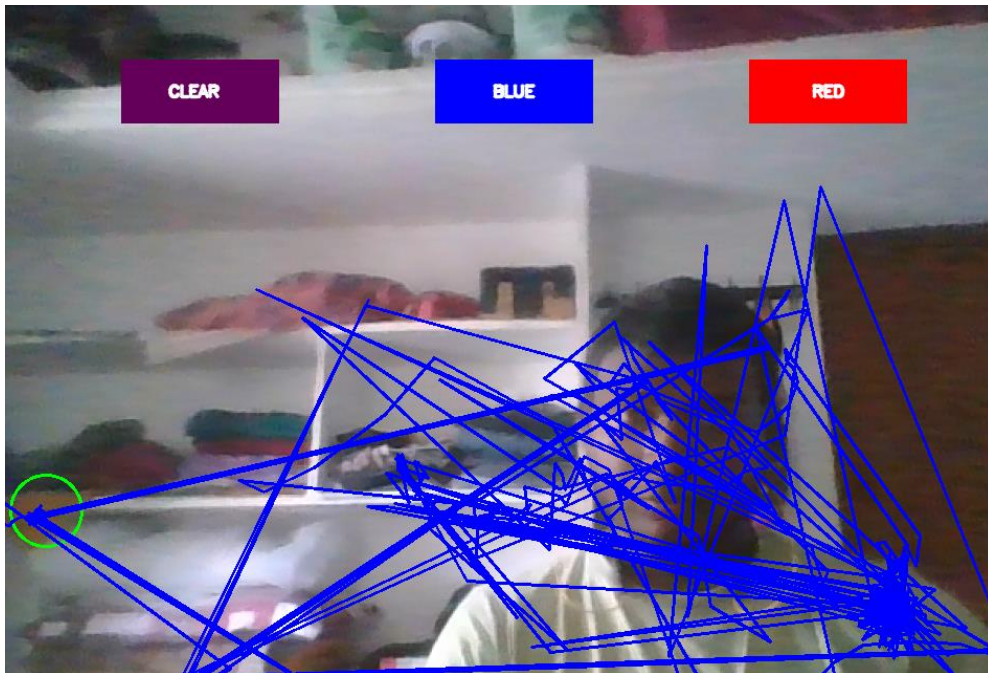
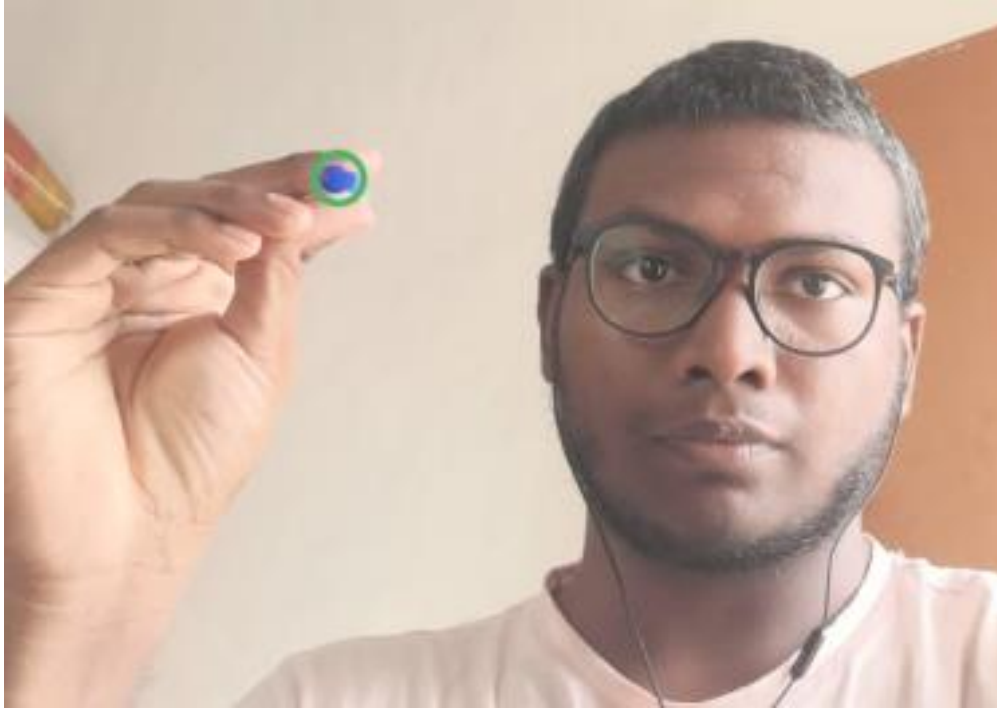


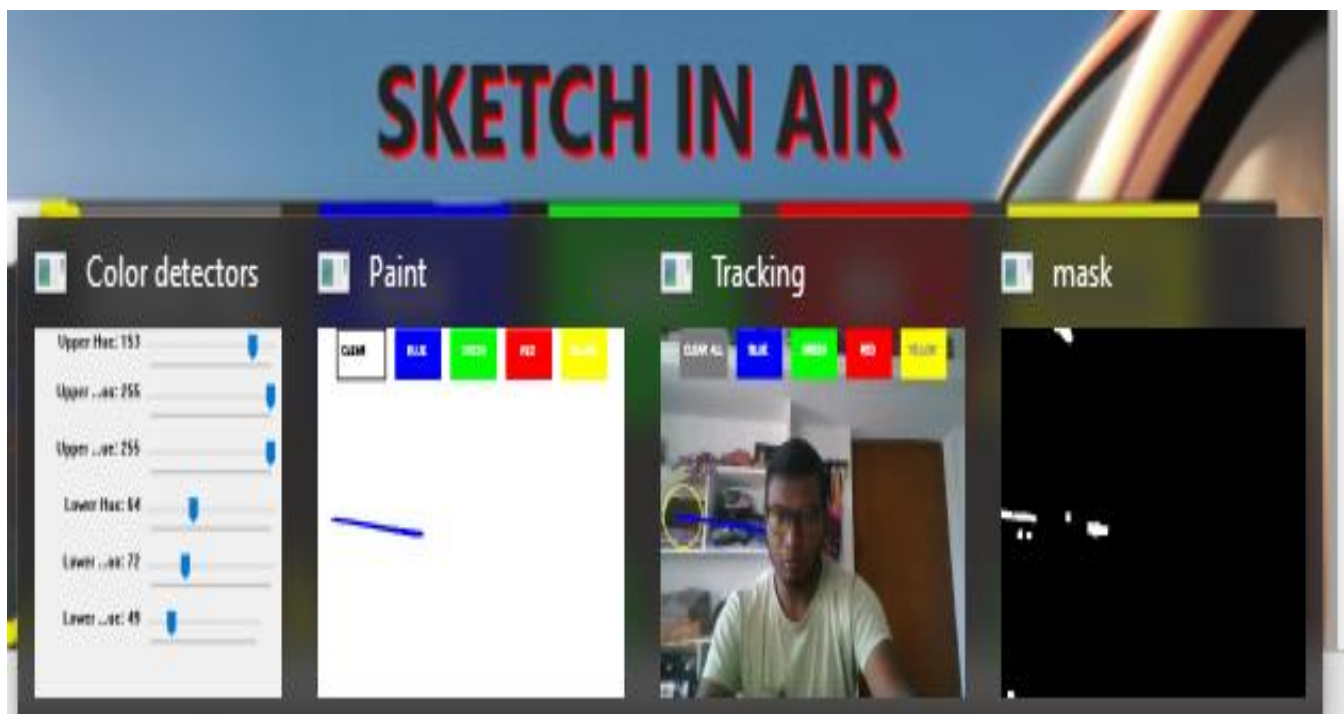
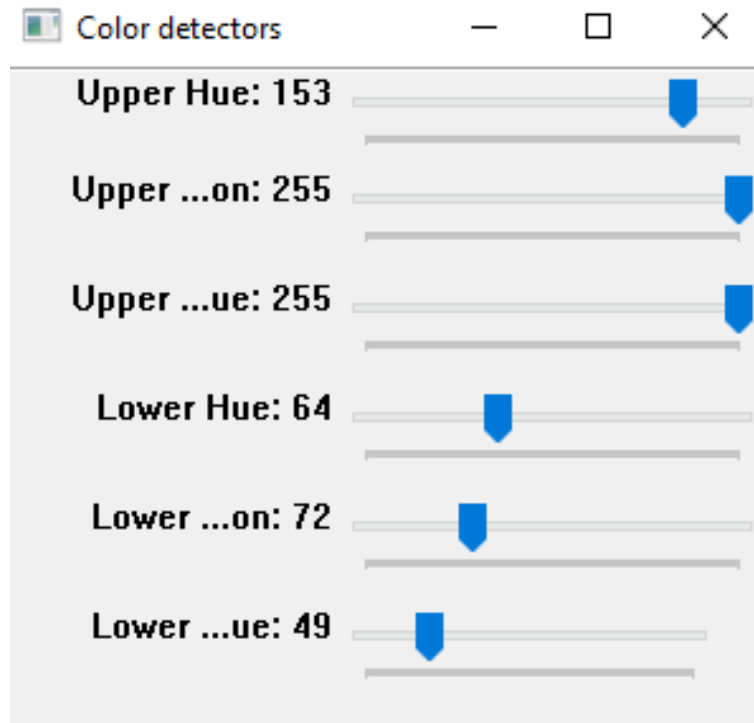
Color detection:

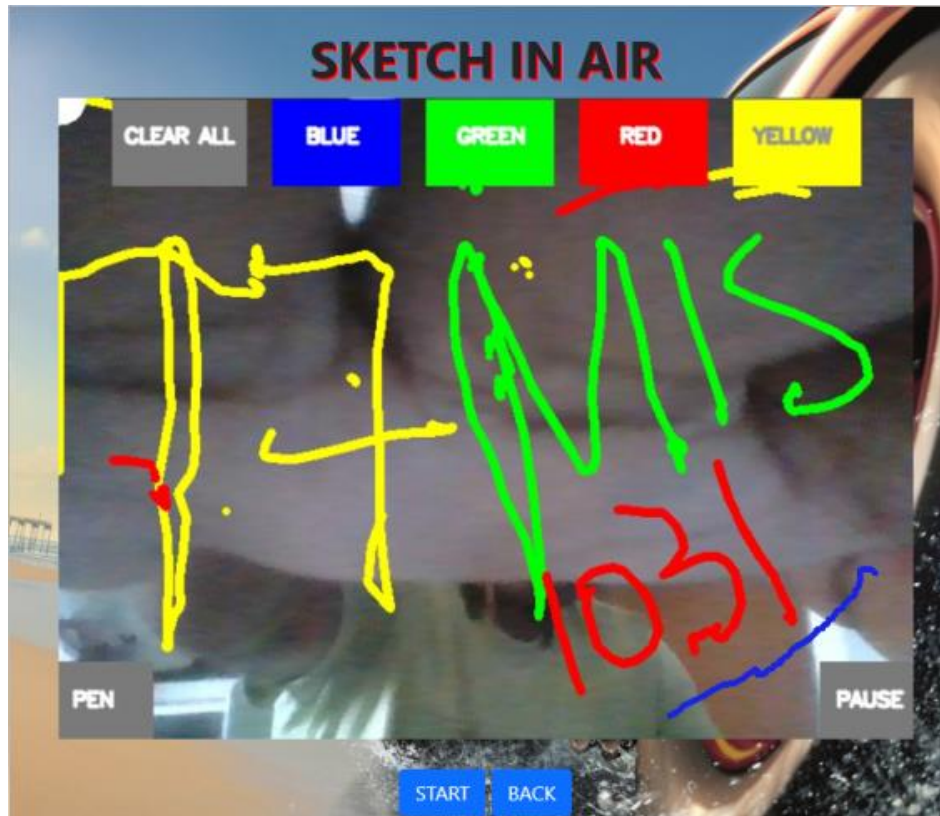




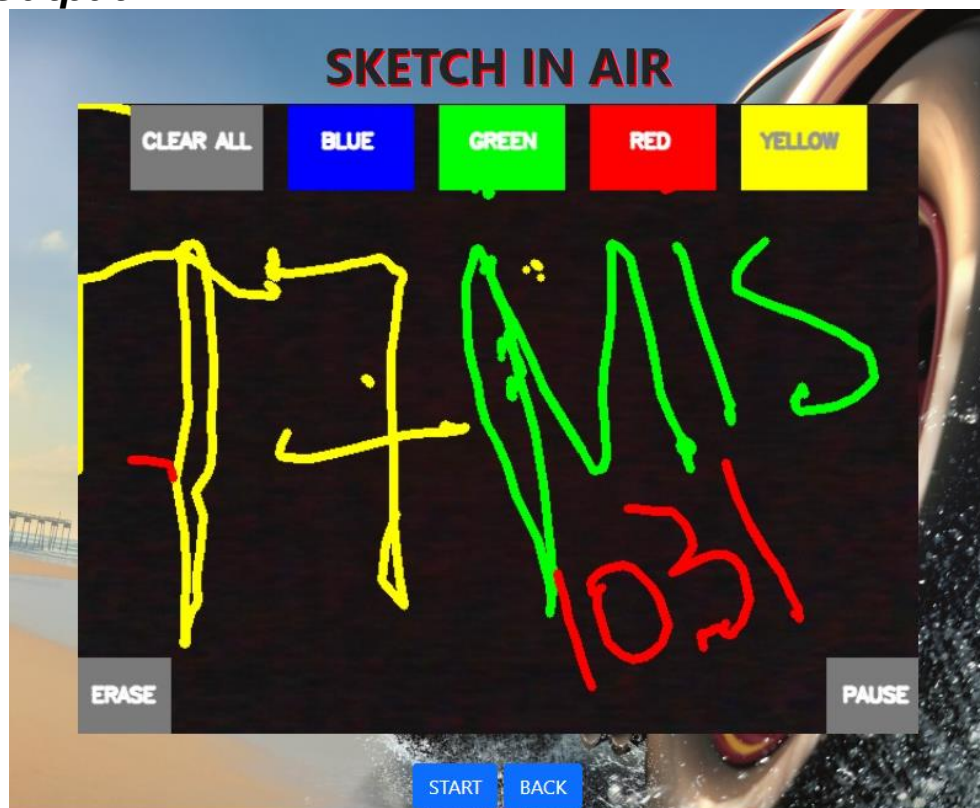
Color Tip Detection:







Final Output:



Source Code:

```
1. import cv2 as opencv

2. from keras.models import load_model

3. import numpy as numpy

4. import os

5. from collections import deque

6. from flask import Flask, render_template, Response, redirect,
    url_for

7. import time

8. app=Flask(__name__)

9. def setValues(x):

10. print("")

11. def
    generate_frames(colorIndex, colors, kernel, bpoints, gpoints, rpoints,
    ypoints, bkpoints, blue_index, green_index, red_index, yellow_index,
    black_index):

12. capture = opencv.VideoCapture(0)
```

```
13.drawing = None

14.while True:

15.ret, frame = capture.read()


16.frame = opencv.flip(frame, 1)

17.if drawing is None:

18.drawing = numpy.zeros_like(frame)

19.hsv = opencv.cvtColor(frame, opencv.COLOR_BGR2HSV)


20.u_hue = opencv.getTrackbarPos("Upper Hue", "Color detectors")

21.u_saturation = opencv.getTrackbarPos("Upper Saturation", "Color
    detectors")

22.u_value = opencv.getTrackbarPos("Upper Value", "Color detectors")

23.l_hue = opencv.getTrackbarPos("Lower Hue", "Color detectors")

24.l_saturation = opencv.getTrackbarPos("Lower Saturation", "Color
    detectors")

25.l_value = opencv.getTrackbarPos("Lower Value", "Color detectors")

26.Upper_hsv = numpy.array([u_hue,u_saturation,u_value])
```

```
27. Lower_hsv = numpy.array([l_hue,l_saturation,l_value])

28. frame = opencv.rectangle(frame, (40,1), (140,65), (122,122,122),
    -1)

29. frame = opencv.rectangle(frame, (160,1), (255,65), colors[0], -1)

30. frame = opencv.rectangle(frame, (275,1), (370,65), colors[1], -1)

31. frame = opencv.rectangle(frame, (390,1), (485,65), colors[2], -1)

32. frame = opencv.rectangle(frame, (505,1), (600,65), colors[3], -1)

33. frame = opencv.rectangle(frame, (570,422), (640,480),
    (122,122,122), -1)

34. frame = opencv.rectangle(frame, (0,422), (70,480), (122,122,122),
    -1)

35. opencv.putText(frame, "CLEAR ALL", (49, 33),
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
    opencv.LINE_AA)

36. opencv.putText(frame, "BLUE", (185, 33),
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
    opencv.LINE_AA)
```



```
37. opencv.putText(frame, "GREEN", (298, 33),  
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
    opencv.LINE_AA)  
  
38. opencv.putText(frame, "RED", (420, 33),  
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
    opencv.LINE_AA)  
  
39. opencv.putText(frame, "YELLOW", (520, 33),  
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (150,150,150), 2,  
    opencv.LINE_AA)  
  
40. opencv.putText(frame, "PAUSE", (582, 455),  
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
    opencv.LINE_AA)  
  
41. opencv.putText(frame, "ERASE", (10, 455),  
    opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
    opencv.LINE_AA)  
  
42. Mask = opencv.inRange(hsv, Lower_hsv, Upper_hsv)
```

```
43.Mask = opencv.erode(Mask, kernel, iterations=1)

44.Mask = opencv.morphologyEx(Mask, opencv.MORPH_OPEN, kernel)

45.Mask = opencv.dilate(Mask, kernel, iterations=1)


46.cnts,_ = opencv.findContours(Mask.copy(),

    opencv.RETR_EXTERNAL,opencv.CHAIN_APPROX_SIMPLE)

47.center = None


48.if len(cnts) > 0:


49.cnt = sorted(cnts, key = opencv.contourArea, reverse = True)[0]

50.((x, y), radius) = opencv.minEnclosingCircle(cnt)

51.opencv.circle(frame, (int(x), int(y)), int(radius), (0, 255,

    255), 2)

52.M = opencv.moments(cnt)

53.center = (int(M['m10'] / M['m00']), int(M['m01'] / M['m00']))

54.if center[1] >= 422 and center[0] >= 570:

55.break
```

```
56. if center[1] >= 422 and center[0] <= 70:
```

```
57. colorIndex = 4
```

```
58. if center[1] <= 65:
```

```
59. if 40 <= center[0] <= 140:
```

```
60. bpoints = [deque(maxlen=512)]
```

```
61. gpoints = [deque(maxlen=512)]
```

```
62. rpoints = [deque(maxlen=512)]
```

```
63. ypoints = [deque(maxlen=512)]
```

```
64. bkpoints = [deque(maxlen=512)]
```

```
65. blue_index = 0
```

```
66. green_index = 0
```

```
67. red_index = 0
```

```
68. yellow_index = 0
```

```
69. drawing[67:,:,:] = 0
```

```
70. elif 160 <= center[0] <= 255:
```

```
71. colorIndex = 0
```

```
72.elif 275 <= center[0] <= 370:

73.colorIndex = 1

74.elif 390 <= center[0] <= 485:

75.colorIndex = 2

76.elif 505 <= center[0] <= 600:

77.colorIndex = 3

78.else :

79.if colorIndex == 0:

80.bpoints[blue_index].appendleft(center)

81.elif colorIndex == 1:

82.gpoints[green_index].appendleft(center)

83.elif colorIndex == 2:

84.rpoints[red_index].appendleft(center)

85.elif colorIndex == 3:

86.ypoints[yellow_index].appendleft(center)

87.else:

88.bpoints.append(deque(maxlen=512))

89.blue_index += 1
```

School of Computer Science & Engineering, Vellore Institute of Technology, Chennai

```
104.     255, opencv.THRESH_BINARY)

105.     foreground = opencv.bitwise_and(drawing, drawing, mask =
        mask)

106.     background = opencv.bitwise_and(frame, frame,

107.     mask = opencv.bitwise_not(mask))

108.     frame = opencv.add(foreground,background)


109.     # opencv.imshow("Tracking", drawing)

110.     # opencv.imshow("Paint", paintWindow)

111.     opencv.imshow("mask",Mask)

112.     ret, buffer = opencv.imencode('.jpg', frame)

113.     frame = buffer.tobytes()

114.     yield (b'--frame\r\n'

115.     b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n') #

        concat frame one by one and show result


116.     if opencv.waitKey(1) & 0xFF == ord("q"):

117.         break

118.     capture.release()
```

```
119.     opencv.destroyAllWindows()

120.     #def setValues(x):

121.     #   print("")

122.     def updated_generate_frames():

123.         load_from_disk = True

124.         if load_from_disk:

125.             penval = numpy.load('penval.npy')

126.             capture = opencv.VideoCapture(0)

127.             # Load these 2 images and resize them to the same size.

128.             # pen_img = opencv.resize(opencv.imread('air drawing
            files/pen.png',1), (50, 50))

129.             # eraser_img = opencv.resize(opencv.imread('air drawing
            files/eraser.jpg',1), (50, 50))
```

```
130.     kernel = numpy.ones((5,5),numpy.uint8)

131.     # Making window size adjustable

132.     opencv.namedWindow('image', opencv.WINDOW_NORMAL)

133.     # This is the drawing on which we will draw upon

134.     drawing = None

135.     # Create a background subtractor Object

136.     backgroundobject =

        opencv.createBackgroundSubtractorMOG2(detectShadows = False)

137.     # This threshold determines the amount of disruption in the

        background.

138.     background_threshold = 600

139.     # A variable which tells you if you're using a pen or an

        eraser.
```



```
140.    switch/link/link = 'Pen'

141.    # With this variable we will monitor the time between
        previous switch/link/link.

142.    last_switch/link/link = time.time()

143.    # Initilize x1,y1 points

144.    x1,y1=0,0

145.    # Threshold for noise

146.    noiseth = 800

147.    # Threshold for wiper, the size of the contour must be
        bigger than this for # us to clear the drawing

148.    wiper_thresh = 40000

149.    # A variable which tells when to clear drawing

150.    clear = False
```

```
151.     colorIndex=0

152.     while(1):

153.         _, frame = capture.read()

154.         frame = opencv.flip( frame, 1 )


155.         # Initilize the drawing as a black image

156.         if drawing is None:

157.             drawing = numpy.zeros_like(frame)


158.         colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (0, 255,
                255), (0,0,0)]


159.         # Take the top left of the frame and apply the background
            subtractor

160.         # there

161.         # top_left = frame[422: 480, 0: 70]

162.         # fgmask = backgroundobject.apply(top_left)
```

```
163.      # # Note the number of pixels that are white, this is the
          level of
164.      # # disruption.
165.      # switch/link/link_thresh = numpy.sum(fgmask==255)

166.      # # If the disruption is greater than background threshold
          and there has
167.      # # been some time after the previous switch/link/link then
          you. can change the
168.      # # object type.
169.      # if switch/link/link_thresh>background_threshold and
          (time.time()-last_switch/link/link) > 1:

170.      #      # Save the time of the switch/link/link.
171.      #      last_switch/link/link = time.time()

172.      #      if switch/link/link == 'Pen':
```

```
173.      #          switch/link/link = 'Eraser'

174.      #      else:

175.      #          switch/link/link = 'Pen'


176.      # Modify BGR to HSV

177.      hsv = opencv.cvtColor(frame, opencv.COLOR_BGR2HSV)


178.      # If you're reading from memory then load the upper and
        lower ranges

179.      # from there

180.      if load_from_disk:

181.          lower_range = penval[0]

182.          upper_range = penval[1]


183.      # Otherwise define your own custom values for upper and
        lower range.

184.      else:

185.          lower_range = numpy.array([26,80,147])
```

```
186.     upper_range = numpy.array([81,255,255])

187.     mask = opencv.inRange(hsv, lower_range, upper_range)

188.     # Perform morphological operations to get rid of the noise

189.     mask = opencv.erode(mask,kernel,iterations = 1)

190.     mask = opencv.dilate(mask,kernel,iterations = 2)


191.     # Find Contours

192.     contours, _ = opencv.findContours(mask,

        opencv.RETR_EXTERNAL,

193.     opencv.CHAIN_APPROX_SIMPLE)


194.     # Make sure there is a contour present and also its size is

        bigger than

195.     # noise threshold.

196.     if contours and opencv.contourArea(max(contours,

197.     key = opencv.contourArea)) > noiseth:
```

```
198.     c = max(contours, key = opencv.contourArea)

199.     x2,y2,w,h = opencv.boundingRect(c)


200.     # Get the area of the contour

201.     area = opencv.contourArea(c)


202.     # If there were no previous points then save the detected
        x2,y2

203.     # coordinates as x1,y1.

204.     center=[x2, y2]

205.     if center[1] >= 422 and center[0] >= 570:

206.         break

207.     elif (center[1] >= 422 and center[0] <= 70) and
        (time.time()-last_switch/link/link) > 1:

208.         last_switch/link/link = time.time()

209.         if switch/link/link == 'Pen':

210.             switch/link/link = 'Eraser'
```

```
211.     else:

212.         switch/link/link = 'Pen'

213.         if center[1] <= 65:

214.             if 40 <= center[0] <= 140:

215.                 drawing[:, :, :] = 0

216.             elif 160 <= center[0] <= 255:

217.                 colorIndex = 0

218.             elif 275 <= center[0] <= 370:

219.                 colorIndex = 1

220.             elif 390 <= center[0] <= 485:

221.                 colorIndex = 2

222.             elif 505 <= center[0] <= 600:

223.                 colorIndex = 3

224.         if x1 == 0 and y1 == 0:

225.             x1,y1= x2,y2

226.         else:

227.             if switch/link/link == 'Pen':
```

```
228.     # Draw the line on the drawing

229.     drawing = opencv.line(drawing, (x1,y1), (x2,y2),

        colors[colorIndex], 5)

230.     else:

231.         opencv.circle(drawing, (x2, y2), 20, (0,0,0), -1)

232.     # After the line is drawn the new points become the previous
        points.

233.     x1,y1 = x2,y2

234.     else:

235.         # If there were no contours detected then make x1,y1 = 0

236.         x1,y1 = 0,0

237.     # Now this piece of code is just for smooth drawing.

        (Optional)

238.     _ , mask = opencv.threshold(opencv.cvtColor (drawing,

        opencv.COLOR_BGR2GRAY), 20,

239.     255, opencv.THRESH_BINARY)
```



```
240.     foreground = opencv.bitwise_and(drawing, drawing, mask =  
        mask)  
  
241.     background = opencv.bitwise_and(frame, frame,  
  
242.     mask = opencv.bitwise_not(mask))  
  
243.     frame = opencv.add(foreground,background)  
  
  
244.     frame = opencv.rectangle(frame, (40,1), (140,65),  
        (122,122,122), -1)  
  
245.     frame = opencv.rectangle(frame, (160,1), (255,65),  
        colors[0], -1)  
  
246.     frame = opencv.rectangle(frame, (275,1), (370,65),  
        colors[1], -1)  
  
247.     frame = opencv.rectangle(frame, (390,1), (485,65),  
        colors[2], -1)  
  
248.     frame = opencv.rectangle(frame, (505,1), (600,65),  
        colors[3], -1)  
  
249.     frame = opencv.rectangle(frame, (570,422), (640,480),  
        (122,122,122), -1)
```

```
250.     frame = opencv.rectangle(frame, (0,422), (70,480),  
  
        (122,122,122), -1)  
  
251.     opencv.putText(frame, "CLEAR ALL", (49, 33),  
  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
  
        opencv.LINE_AA)  
  
252.     opencv.putText(frame, "BLUE", (185, 33),  
  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
  
        opencv.LINE_AA)  
  
253.     opencv.putText(frame, "GREEN", (298, 33),  
  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
  
        opencv.LINE_AA)  
  
254.     opencv.putText(frame, "RED", (420, 33),  
  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
  
        opencv.LINE_AA)  
  
255.     opencv.putText(frame, "YELLOW", (520, 33),  
  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (150,150,150), 2,  
  
        opencv.LINE_AA)
```

```
256.     opencv.putText(frame, "PAUSE", (582, 455),  
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
        opencv.LINE_AA)  
  
257.     if(switch/link/link == 'Pen'):  
  
258.         opencv.putText(frame, "ERASE", (10, 455),  
            opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
            opencv.LINE_AA)  
  
259.         else:  
  
260.             opencv.putText(frame, "PEN", (10, 455),  
                opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,  
                opencv.LINE_AA)  
  
  
261.         if switch/link/link != 'Pen':  
  
262.             opencv.circle(frame, (x1, y1), 20, (255,255,255), -1)  
  
  
263.         ret, buffer = opencv.imencode('.jpg', frame)  
  
264.         frame = buffer.tobytes()  
  
265.         yield (b'--frame\r\n')
```

```
266.      b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')  #
        concat frame one by one and show result

267.      k = opencv.waitKey(5) & 0xFF

268.      if k == 27:

269.          break

270.      # Clear the drawing after 1 second, if the clear variable is
        true

271.      if clear == True:

272.          time.sleep(1)

273.          drawing = None

274.      # And then set clear to false

275.      clear = False

276.      opencv.destroyAllWindows()

277.      capture.release()
```

```
278.     @app.route('/')

279.     def index():

280.         return render_template('index.html')


281.     @app.route('/video')

282.     def video():

283.         return render_template('video.html')


284.     @app.route('/video_feed')

285.     def video_feed():

286.         print("inside video -----")

287.         # opencv.namedWindow("Color detectors")

288.         # opencv.createTrackbar("Upper Hue", "Color detectors", 162,

180,setValues)

289.         # opencv.createTrackbar("Upper Saturation", "Color

detectors", 255, 255,setValues)
```

```
290.     # opencv.createTrackbar("Upper Value", "Color detectors",
        255, 255,setValues)

291.     # opencv.createTrackbar("Lower Hue", "Color detectors", 88,
        180,setValues)

292.     # opencv.createTrackbar("Lower Saturation", "Color
        detectors", 108, 255,setValues)

293.     # opencv.createTrackbar("Lower Value", "Color detectors",
        76, 255,setValues)


294.     # bpoints = [deque(maxlen=1024)]

295.     # gpoints = [deque(maxlen=1024)]

296.     # rpoints = [deque(maxlen=1024)]

297.     # ypoints = [deque(maxlen=1024)]

298.     # bkpoints = [deque(maxlen=1024)]


299.     # blue_index = 0

300.     # green_index = 0

301.     # red_index = 0
```

```
302.     # yellow_index = 0

303.     # black_index = 0


304.     # kernel = numpy.ones((5,5),numpy.uint8)


305.     # colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (0, 255,
        255), (0,0,0)]

306.     # colorIndex = 0


307.     # paintWindow = numpy.zeros((471,636,3)) + 255

308.     # paintWindow = opencv.rectangle(paintWindow, (40,1),
        (140,65), (0,0,0), 2)

309.     # paintWindow = opencv.rectangle(paintWindow, (160,1),
        (255,65), colors[0], -1)

310.     # paintWindow = opencv.rectangle(paintWindow, (275,1),
        (370,65), colors[1], -1)

311.     # paintWindow = opencv.rectangle(paintWindow, (390,1),
        (485,65), colors[2], -1)
```

```
312.      # paintWindow = opencv.rectangle(paintWindow, (505,1),
      (600,65), colors[3], -1)

313.      # opencv.putText(paintWindow, "CLEAR", (49, 33),
      opencv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2, opencv.LINE_AA)

314.      # opencv.putText(paintWindow, "BLUE", (185, 33),
      opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
      opencv.LINE_AA)

315.      # opencv.putText(paintWindow, "GREEN", (298, 33),
      opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
      opencv.LINE_AA)

316.      # opencv.putText(paintWindow, "RED", (420, 33),
      opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
      opencv.LINE_AA)

317.      # opencv.putText(paintWindow, "YELLOW", (520, 33),
      opencv.FONT_HERSHEY_SIMPLEX, 0.5, (150,150,150), 2,
      opencv.LINE_AA)

318.      # opencv.namedWindow('Paint', opencv.WINDOW_AUTOSIZE)
```



```
319.     # capture = opencv.VideoCapture(0)

320.     print("going to generate frames function")

321.     #

        generate_frames(colorIndex,paintWindow,colors,kernel,bpoints,gpoi
        nts,rpoints,ypoints,blue_index,green_index,red_index,yellow_index
        )

322.     # return redirect(url_for('index'))

323.     # return

        Response(generate_frames(colorIndex,colors,kernel,bpoints,gpoints
        ,rpoints,ypoints,bkpoints,blue_index,green_index,red_index,yellow
        _index, black_index),mimetype='multipart/x-mixed-replace;
        boundary=frame')

324.     return

        Response(updated_generate_frames(),mimetype='multipart/x-mixed-
        replace; boundary=frame')
```

```
325.     #EMOJI

326.     def keras_predict(model, image):

327.         processed = keras_process_image(image)

328.         pred_probab = model.predict(processed)[0]

329.         pred_class = list(pred_probab).index(max(pred_probab))

330.         return max(pred_probab), pred_class


331.     def keras_process_image(img):

332.         image_x = 50

333.         image_y = 50

334.         img = opencv.resize(img, (image_x, image_y))

335.         img = numpy.array(img, dtype=numpy.float32)

336.         img = numpy.reshape(img, (-1, image_x, image_y, 1))

337.         return img
```

```
338.     def get_emojis():
339.         emojis_folder = 'emojinator_files/hand_emo/'
340.         emojis = []
341.         for emoji in range(len(os.listdir(emojis_folder))):
342.             print(emoji)
343.             emojis.append(opencv.imread(emojis_folder + str(emoji) +
                '.png', -1))
344.         return emojis

345.     def overlay(image, emoji, x, y, w, h):
346.         emoji = opencv.resize(emoji, (w, h))
347.         try:
348.             image[y:y + h, x:x + w] = blend_transparent(image[y:y + h,
                x:x + w], emoji)
349.         except:
350.             pass
```

```
351.     return image

352.     def blend_transparent(face_img, overlay_t_img):

353.         # Split out the transparency mask from the colour info

354.         overlay_img = overlay_t_img[:, :, :3] # Grab the BRG planes

355.         overlay_mask = overlay_t_img[:, :, 3:] # And the alpha
           plane

356.         # Again calculate the inverse mask

357.         background_mask = 255 - overlay_mask

358.         # Turn the masks into three channel, so we can use them as
           weights

359.         overlay_mask = opencv.cvtColor(overlay_mask,
           opencv.COLOR_GRAY2BGR)

360.         background_mask = opencv.cvtColor(background_mask,
           opencv.COLOR_GRAY2BGR)
```

```
361.     # Create a masked out face image, and masked out overlay

362.     # We modify the images to floating point in range 0.0 - 1.0

363.     face_part = (face_img * (1 / 255.0)) * (background_mask * (1

        / 255.0))

364.     overlay_part = (overlay_img * (1 / 255.0)) * (overlay_mask *

        (1 / 255.0))

365.     # And finally just add them together, and rescale it back to

        an 8bit integer image

366.     return numpy.uint8(opencv.addWeighted(face_part, 255.0,

        overlay_part, 255.0, 0.0))

367.     def generate_emoji(capture, detection_graph, sess,

        num_hands_detect, im_width, im_height, model, emojis):

368.         while True:

369.             # Expand dimensions since the model expects images to have

                shape: [1, None, None, 3]
```

```
370.     ret, image_numpy = capture.read()

371.     image_numpy = opencv.flip(image_numpy, 1)

372.     # image_numpy = opencv.flip(image_numpy, 1)

373.     try:

374.         image_numpy = opencv.cvtColor(image_numpy,

                                         opencv.COLOR_BGR2RGB)

375.     except:

376.         print("Error modifying to RGB")


377.     # Actual detection. Variable boxes contains the bounding box

        coordinates for hands detected,

378.     # while scores contains the confidence for each of these

        boxes.

379.     # Hint: If len(boxes) > 1 , you may assume you have found

        atleast one hand (within your score threshold)


380.     boxes, scores = detector_utils.detect_objects(image_numpy,

381.     detection_graph, sess)
```

```
382.     # draw bounding boxes on frame

383.     img = detector_utils.draw_box_on_image(num_hands_detect,

        0.4,

384.     scores, boxes, im_width, im_height,

385.     image_numpy)

386.     image_numpy=opencv.cvtColor(image_numpy,

        opencv.COLOR_RGB2BGR)

387.     img = opencv.cvtColor(img, opencv.COLOR_RGB2BGR)

388.     hsv = opencv.cvtColor(img, opencv.COLOR_BGR2HSV)

389.     mask2 = opencv.inRange(hsv, numpy.array([2, 50, 60]),

        numpy.array([25, 150, 255]))

390.     res = opencv.bitwise_and(img, img, mask=mask2)

391.     gray = opencv.cvtColor(res, opencv.COLOR_BGR2GRAY)

392.     median = opencv.GaussianBlur(gray, (5, 5), 0)

393.     kernel_square = numpy.ones((5, 5), numpy.uint8)
```

```
394.     dilation = opencv.dilate(median, kernel_square,
                                   iterations=2)

395.     opening = opencv.morphologyEx(dilation, opencv.MORPH_CLOSE,
                                       kernel_square)

396.     ret, thresh = opencv.threshold(opening, 30, 255,
                                       opencv.THRESH_BINARY)

397.     newImage = opencv.resize(thresh, (50, 50))

398.     pred_probab, pred_class = keras_predict(model, newImage)

399.     print(pred_class, pred_probab)

400.     # if(pred_class==8):

401.     #     break

402.     image_numpy = overlay(image_numpy, emojis[pred_class], 400,
                             300, 90, 90)

403.     # image_numpy = opencv.rectangle(image_numpy, (570,422),
                                       (640,478), (122,122,122), -1)
```



```
404.     # opencv.putText(image_numpy, "PAUSE", (582, 455),
        opencv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2,
        opencv.LINE_AA)

405.     # opencv.imshow('Single-Threaded Detection',
406.     #                 image_numpy)
407.     # opencv.imshow('img', img)
408.     ret, buffer = opencv.imencode('.jpg', image_numpy)
409.     image_numpy = buffer.tobytes()
410.     yield (b'--frame\r\n'
411.            b'Content-Type: image/jpeg\r\n\r\n' + image_numpy + b'\r\n')

    # concat frame one by one and show result

412.     if opencv.waitKey(25) & 0xFF == ord('q'):
413.         opencv.destroyAllWindows()
414.         break
```

```
415.     model = load_model('emojinator_files/emojinator.h5')

416.     detection_graph, sess =

        detector_utils.load_inference_graph()


417.     keras_predict(model, numpy.zeros((50, 50, 1),

        dtype=numpy.uint8))


418.     @app.route('/emoji')

419.     def emoji():

420.         return render_template('emoji.html')


421.     @app.route('/emoji_main')

422.     def emoji_main():


423.         emojis = get_emojis()


424.         capture = opencv.VideoCapture(0)
```

```
425.     capture.set(opencv.CAPTURE_PROP_FRAME_WIDTH, 640)

426.     capture.set(opencv.CAPTURE_PROP_FRAME_HEIGHT, 720)


427.     im_width, im_height = (capture.get(3), capture.get(4))

428.     # max number of hands we want to detect/track

429.     num_hands_detect = 1


430.     # opencv.namedWindow('Single-Threaded Detection',
    opencv.WINDOW_NORMAL)


431.     print("going to generate frames function")

432.     # generate_emoji(capture, detection_graph, sess,
    num_hands_detect, im_width, im_height, model, emojis)

433.     # return redirect(url_for('index'))

434.     return Response(generate_emoji(capture, detection_graph,
    sess, num_hands_detect, im_width, im_height, model,
    emojis), mimetype='multipart/x-mixed-replace; boundary=frame')
```

```
435.     if __name__=="__main__":  
  
436.         print("inside main")  
  
437.         app.run(threaded=False)
```